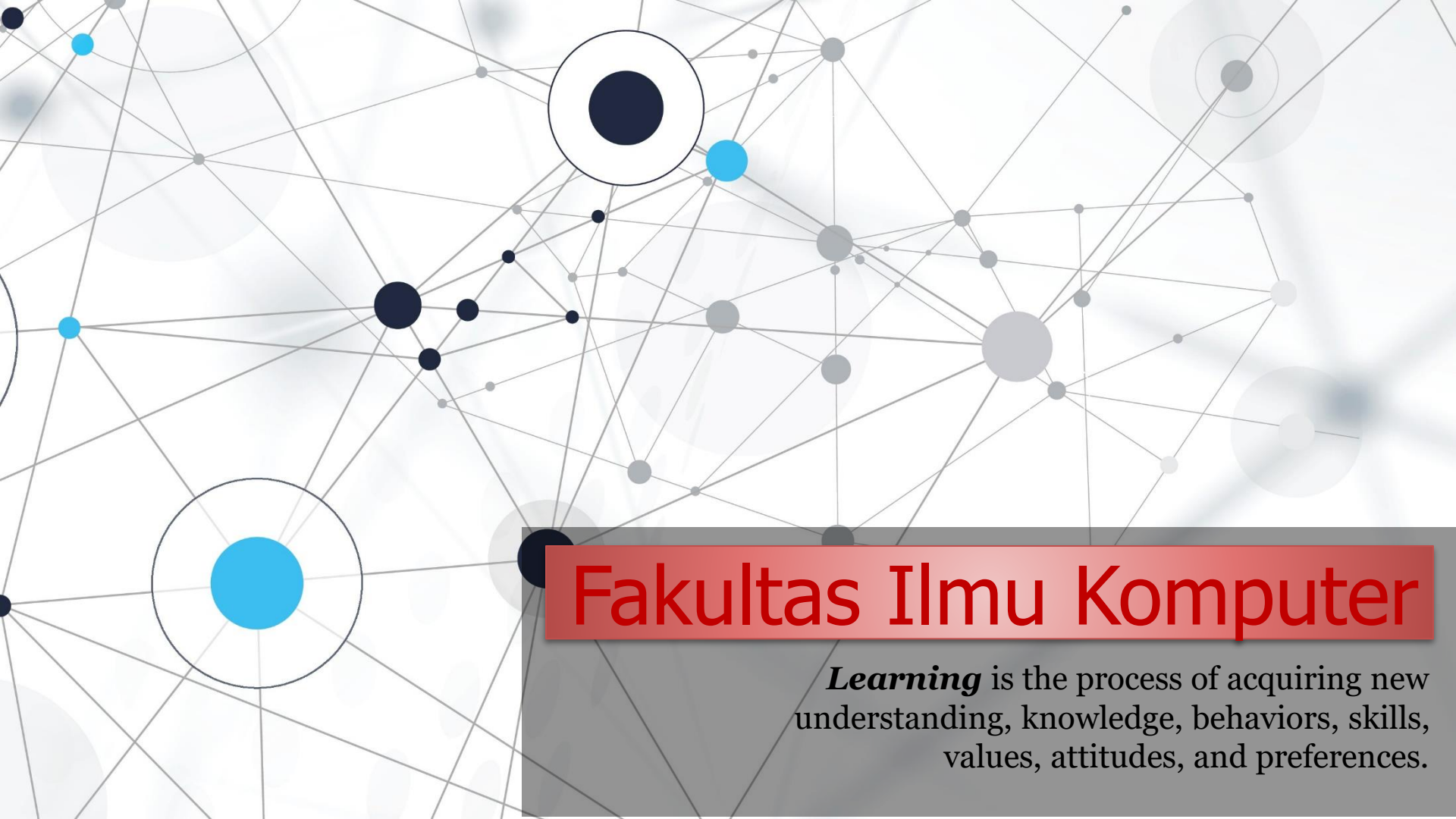


An abstract network diagram with various sized nodes (black, blue, and grey) connected by thin grey lines. Some nodes are highlighted with larger circles. The background is white with faint grey circles.

Fakultas Ilmu Komputer

Learning is the process of acquiring new understanding, knowledge, behaviors, skills, values, attitudes, and preferences.

Constructors and Destructors in Python Class

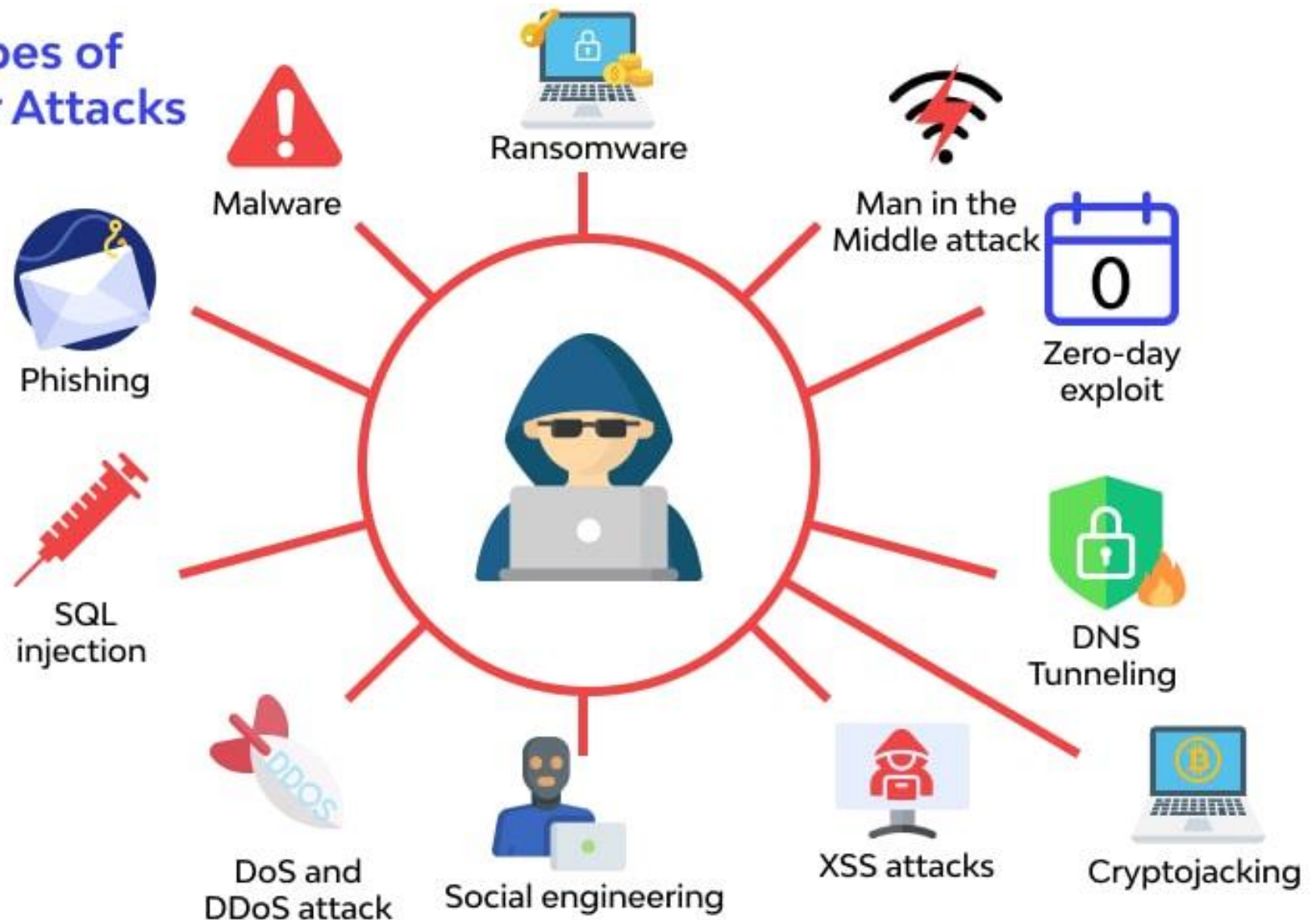


**Welcome to *the* Object Oriented
Programming *course***



Hacking is an activity in which a person (*hacker*), attempts to exploit/utilize (*memanfaatkan*) **weaknesses or security holes** in a computer system or network with the aim of **gaining unauthorized access** or control (**Server**, **Database** dan **Application**). It is possible that hackers (*personal/group*) can access and **modify the content of the website or system** without authorization.

Types of Cyber Attacks



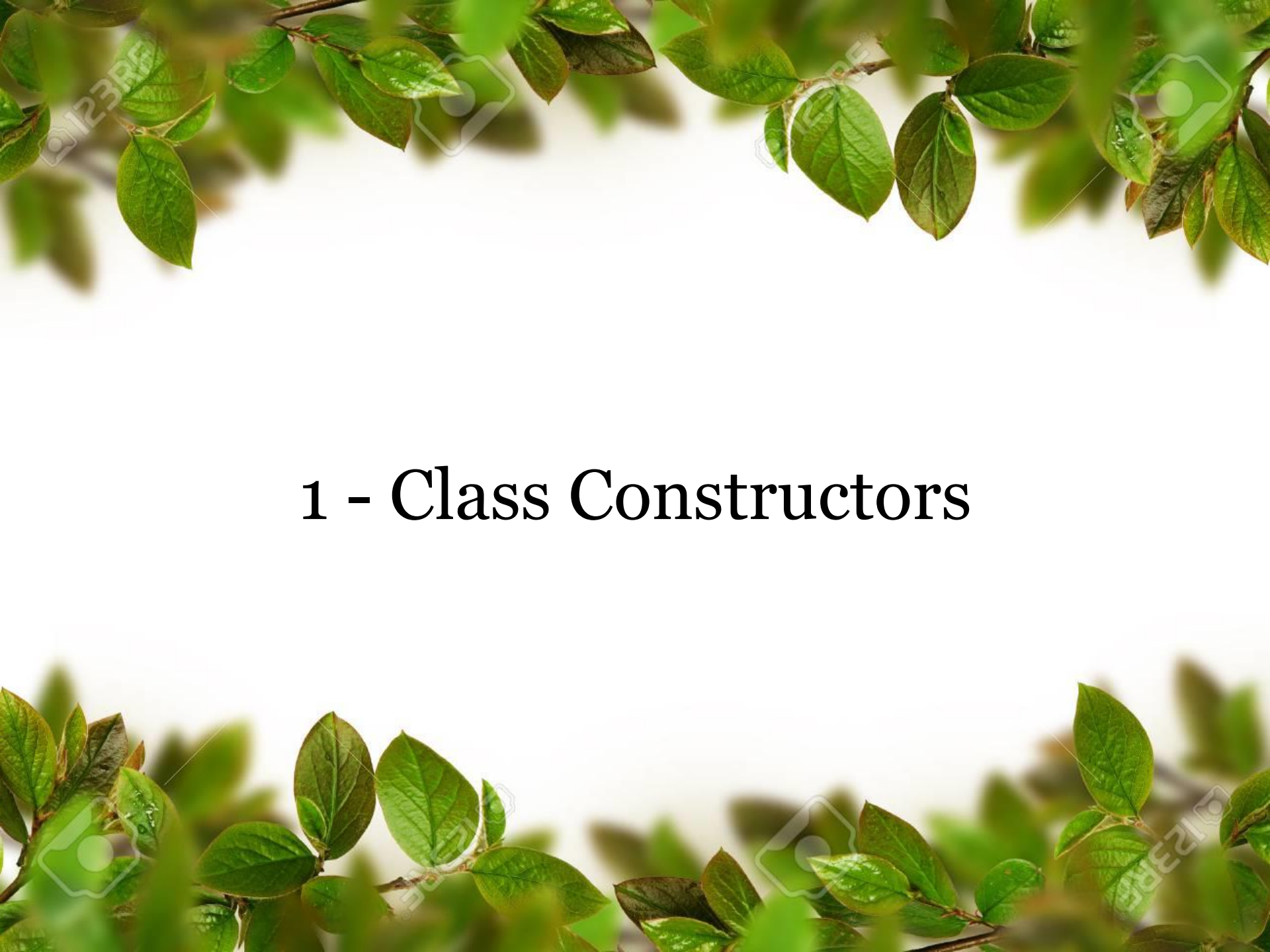
Cross-site Scripting (XSS) is a client-side code injection attack. The attacker aims to execute malicious scripts in a web browser of the victim.

```

1 # create a class
2 class Produk():
3     # variable
4     jumlah_produk = 0
5
6     # method constructor (default method)
7     def __init__(self, nama, harga):
8         self.nama = nama;
9         self.harga = harga;
10
11     # method #1
12     def view_jum_produk(self, jumlah):
13         Produk.jumlah_produk = jumlah;
14         print('Total Produk: ', Produk.jumlah_produk);
15
16     # method #2
17     def detail_produk(self):
18         print("Nama : ", self.nama);
19         print("Harga: ", self.harga);
20
21
22 # membuat objek pertama (Object instantiation)
23 produk1 = Produk('Kerupuk', 5000)
24
25 # membuat objek kedua (Object instantiation)
26 produk2 = Produk('Taro', 3000)
27
28 # membuat objek ketiga (Object instantiation)
29 produk3 = Produk('Astor', 4000)
30
31 # mengakses atribut objek (Accessing class attributes)
32 produk1.detail_produk()
33 produk1.view_jum_produk(5)
34 print("Jumlah produk adalah {}".format(produk1.__class__.jumlah_produk));
35 print("Nama produk adalah {}".format(produk1.nama))
36 print()
37
38 produk2.detail_produk()
39 produk2.view_jum_produk(10)
40 print("Jumlah produk adalah {}".format(produk2.__class__.jumlah_produk));
41 print("Nama produk adalah {}".format(produk2.nama))
42 print()
43
44 produk3.detail_produk()
45 produk3.view_jum_produk(15)
46 print("Jumlah produk adalah {}".format(produk3.__class__.jumlah_produk))
47 print("Nama produk adalah {}".format(produk3.nama))
48

```

1. Sebutkan nama class dari Python file di samping?
2. Sebutkan ada berapa **class variable**?
3. Sebutkan nama-nama method di dalam class tersebut?
4. Sebutkan nama method constructor?
5. Berapa parameter yang terdapat di constructor?
6. Ada berapa object yang telah dibuat dari class Product? Sebutkan?

The slide features a white background with green leaves and branches framing the top and bottom edges. The leaves are vibrant green with some visible veins and slight shadows, creating a natural border around the central text.

1 - Class Constructors

Object Construction or Instantiation

Creating and initializing objects of a given class is a **fundamental step** in object-oriented programming. This step is often referred to as **object construction or instantiation**. The tool responsible for running this instantiation process is commonly known as a **class constructor**.

What is Constructor in Python Class?

```
1 # create class
2 class Student:
3
4     # constructor (initialize instance variable)
5     def __init__(self, name):
6         print('Inside Constructor')
7         self.name = name
8         print('All variables initialized')
9
10    # instance Method
11    def show(self):
12        print('Hello, my name is', self.name)
13
14
15 # create object using constructor
16 s1 = Student('Semmy Wellem Taju')
17
18 # call method
19 s1.show()
```

In object-oriented programming, A constructor (**`def __init__`**) is a special method used to create and initialize an object of a class. This method is defined in the class.

- 1) The **constructor is executed automatically** at the time of object creation.
- 2) The primary use of a constructor is **to declare and initialize data member/instance variables** of a class.
- 3) The constructor contains a collection of statements (*i.e., instructions*) that executes at the time of object creation **to initialize the attributes** of an object.

What is Destructor in Python Class?

In object-oriented programming, A **destructor** (`def __del__`) is called when an object is deleted or destroyed. Destructor is used to perform the clean-up activity before destroying the object, such as closing database connections or *file handle*.

```
# create class
class Student:

    # constructor (initialize instance variable)
    def __init__(self, name):
        print('Inside Constructor')
        self.name = name
        print('All variables initialized')

    # destructor
    def __del__(self):
        print('Inside destructor')
        print('Object destroyed')

    # instance Method
    def show(self):
        print('Hello, my name is', self.name)

# create object using constructor
s1 = Student('Semmy Wellem Taju')
s2 = Student('Eugenie Patricia Lamansiang')
s3 = Student('Putra Budy Harwaiyan')

# call method
s1.show()

# delete object
del s1

# call method
s1.show()
```

- ❖ Python has a **garbage collector** that handles memory management automatically.
- ❖ For example, it cleans up the memory when an object goes out of scope.
- ❖ But it's not just memory that has to be freed when an object is destroyed. **We must release or close the other resources object were using, such as open files, database connections, cleaning up the buffer or cache.** To perform all those cleanup tasks we use **destructor** in Python.

Constructor vs Destructor

- ❖ The **constructor** is used **to initialize objects**, while the **destructor** is used **to delete or destroy the object** that releases the resource occupied by the object.
- ❖ *Constructor (konstruktor) digunakan untuk menginisialisasi objek, sedangkan destructor (destruktor) digunakan untuk menghapus atau menghancurkan objek yang melepaskan sumber daya yang ditempati objek tersebut.*

Constructor	Destructor
Constructors help allocate memory to an object.	Destructors deallocate the memory of an object.
Constructors can take arguments.	Destructors don't take any arguments.
Constructors are called automatically when an object is created.	Destructors are called automatically when the block is exited or when the program terminates.
Constructors allow an object to initialize a value before it is used.	They allow objects to execute code when it is being destroyed.
They are called in the successive order of their creation.	They are called in the reverse order of their creation.
There can be multiple constructors in a single class.	There is a single destructor in a class.
Constructors can be overloaded.	Destructors cannot be overloaded.
The concept of copy constructor allows an object to get initialized from another object.	There is no such concept in the case of destructors.

Constructor *vs* Destructor in Python



constructor

Destructor

`__init__` is used as constructor

`__del__` is used as destructor

used to initialize object's state

used to destroy object's state

called when object is created

called when program ends

Constructor vs Destructor

The destructor is the reverse (*kebalikan*) of the constructor.



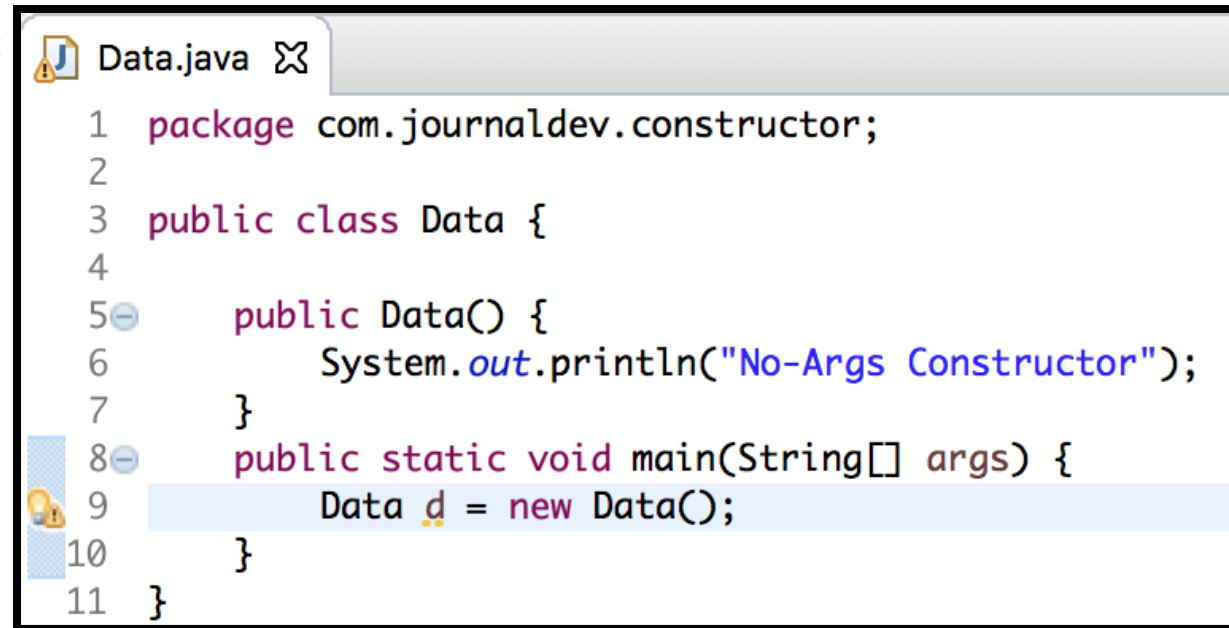
Constructor



Destructor

Constructor & Destructor in Java

```
public class ConstructorExample { ← Class Name
    public ConstructorExample() { ← Constructor Name
    }
    public static void main(String[] args) {
    }
}
```



```
Data.java
1 package com.journaldev.constructor;
2
3 public class Data {
4
5     public Data() {
6         System.out.println("No-Args Constructor");
7     }
8     public static void main(String[] args) {
9         Data d = new Data();
10    }
11 }
```

The **constructor** is used to initialize objects, while the **destructor** is used to delete or destroy the object that releases the resource occupied by the object. Remember that there is no concept of destructor in Java. In place of the destructor, Java provides the **garbage collector** that works the same as the destructor.

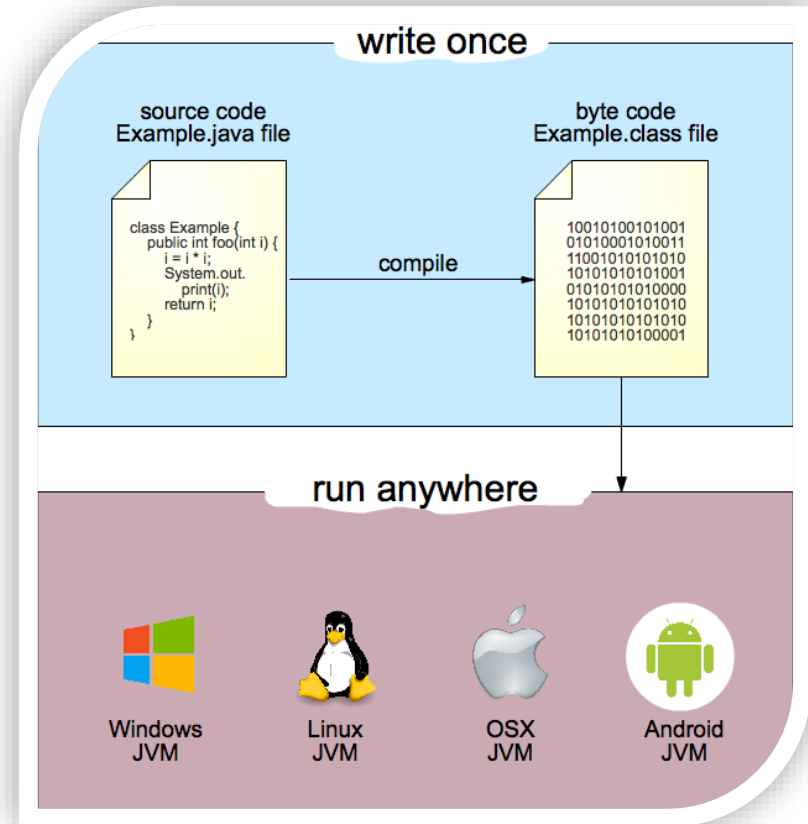
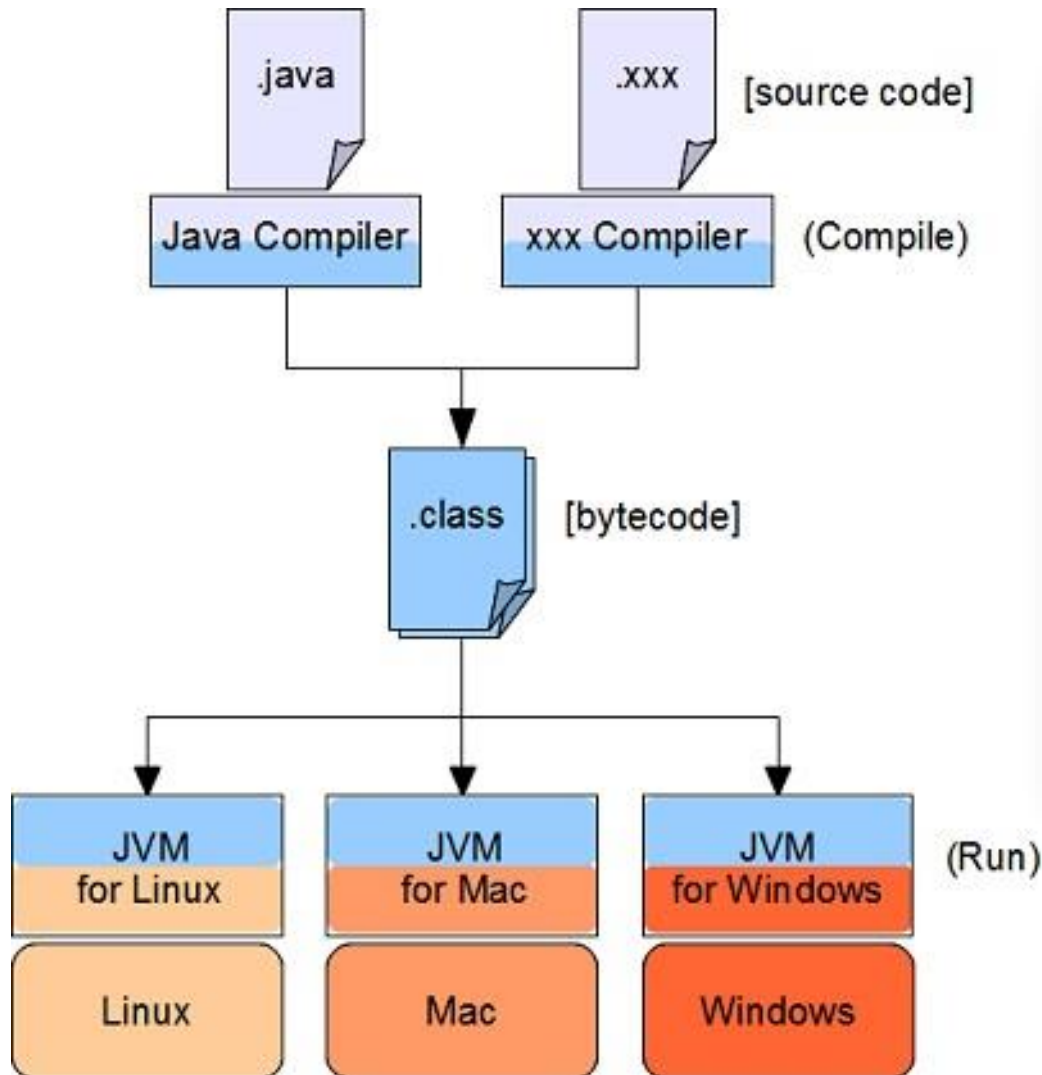
Garbage Collection *in* Java



Garbage Collection is process of reclaiming the runtime unused memory automatically. In other words, it is a way **to destroy the unused objects**. The free() function is used in C language and delete() in C++. But, in Java it is performed automatically. So, java provides better memory management.

- 1) Garbage collection in Java is the process by which Java programs perform **automatic memory management**.
- 2) Java programs compile (.java) to *bytecode* (.class) that can be run on a **Java Virtual Machine**, or JVM for short.
- 3) When **Java programs run on the JVM**, objects are created on the heap, which is a portion of memory dedicated to the program.
- 4) Eventually, some objects will no longer be needed.
- 5) The **garbage collector finds these unused objects** and deletes them to free up memory.

Java Virtual Machine (JVM)



Constructor & Destructor in PHP

```
<?php
class MyDestructableClass {
    function __construct() {
        prin "In constructor\n";
        $this->name = "MyDestructableClass";
    }

    function __destruct() {
        prin "Destroying " . $this->name . "\n";
    }
}

$obj = new MyDestructableClass();
?>
```

```
<?php

class Fun {
    private $name;

    function __construct($name) {
        $this->name = $name;
    }

    function show() {
        echo 'name = ' . $this->name;
    }

    function __destruct() {
        echo "destructor calling!..";
    }
}

$obj = new Fun('master-tech786');
$obj->show();
?>
```

Constructors: Constructors are called when an object is created from a class.

Destructors: Destructors are called when an object destructs.

Usually, it is when the script ends.

Constructor & Destructor in Python

```
# create class
class Student:
    # constructor (initialize instance variable)
    def __init__(self, name):
        print('Inside Constructor')
        self.name = name
        print('All variables initialized')

    # destructor
    def __del__(self):
        print('Inside destructor')
        print('Object destroyed')

    # instance Method
    def show(self):
        print('Hello, my name is', self.name)

# create object using constructor
s1 = Student('Semmy Wellem Taju')
s2 = Student('Eugenie Patricia Lamansiang')
s3 = Student('Putra Budy Harwaiyan')

# call method
s1.show()

# delete object
del s1

# call method
s1.show()
```

The constructor is used to initialize objects, while the destructor is used to delete or destroy the object that releases the resource occupied by the object.

Python's Instantiation Process

```
# create class
class Teacher:
    # instance creator (new empty object)
    def __new__(cls, *args, **kwargs):
        print("1. Create a new instance of class Teacher.")
        return super().__new__(cls)

    # constructor (instance initializer)
    def __init__(self, name, age):
        self.name = name
        self.age = age
        print("2. Initialize the new instance of class Teacher.")
        print('My name is {0} ({1} years old)'.format(self.name, self.age));

    # string representator
    def __repr__(self) -> str:
        return f"{type(self).__name__}(name={self.name}, age={self.age})"

    # destructor (object is deleted or destroyed)
    def __del__(self):
        print('3. Object destroyed.')

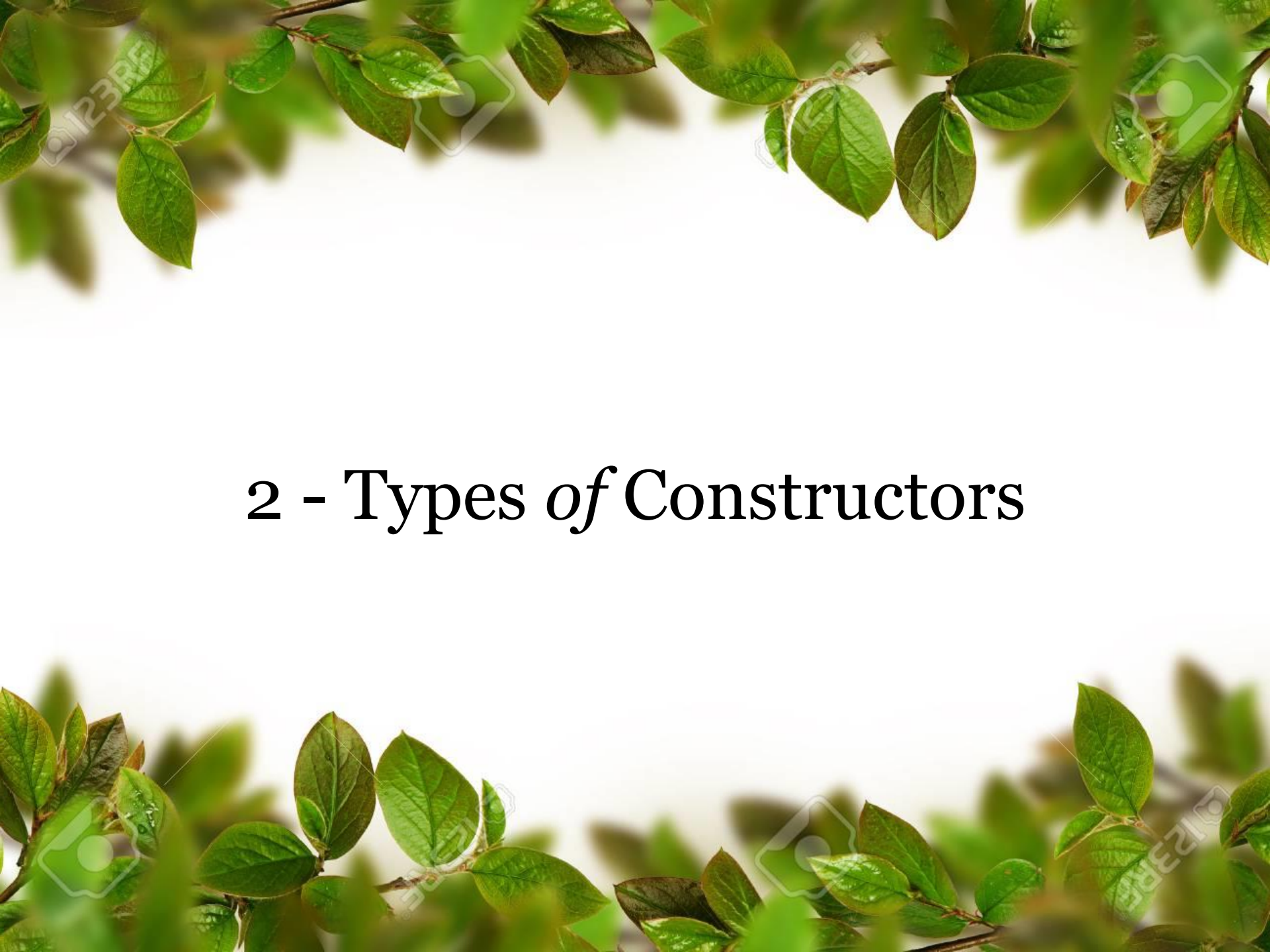
# create object using constructor
t1 = Teacher('Semmy Taju', 30)
t2 = Teacher('Eugenie Lamansiang', 17)

# delete object
del t1
```

When the class constructor is called, which triggers

- 1) The instance creator, `__new__()`, to create a new empty object.
- 2) The instance initializer, `__init__()`, which takes the constructor's arguments to initialize the newly created object.
- 3) The `__repr__()` special method, which provides a proper string **representation** for the class.

Special method called `__new__()`, which is responsible for creating and returning a new empty object. Then another special method, `__init__()`, takes the resulting object, along with the class constructor's arguments.

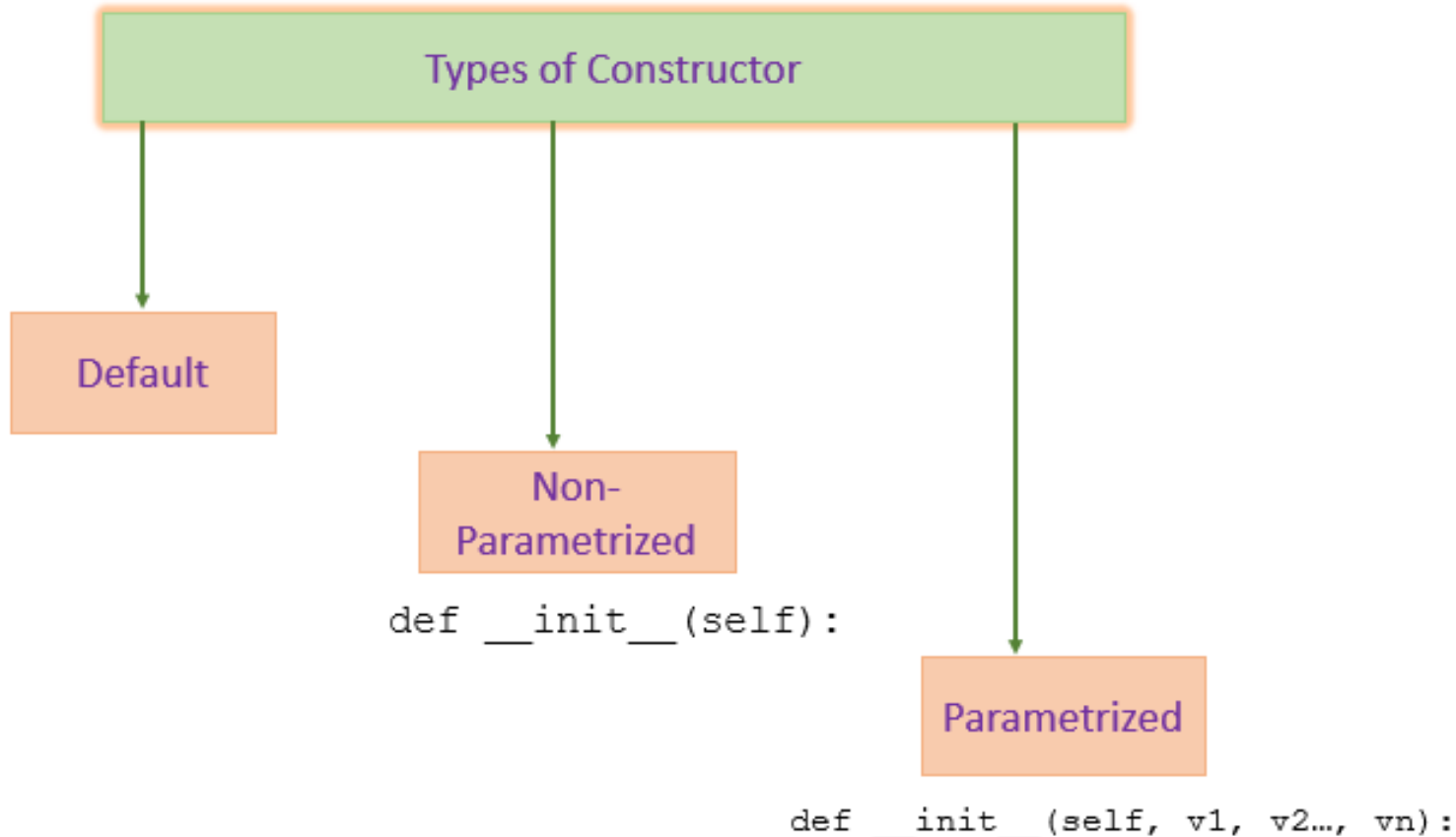
The slide features a white background with green leaves and branches framing the top and bottom edges. The leaves are vibrant green and appear to be from a tree or shrub. The text is centered in the middle of the slide.

2 - Types *of* Constructors

Types of Constructors

In Python, there are three types of constructors:

- 1) **Default Constructor**
- 2) **Non-parameterized constructor**
- 3) **Parameterized constructor**



Default Constructor

```
1  # create class
2  class Employee:
3
4      # display object
5      def showInfo(self):
6          print("This is the information of employee.")
7
8  # creating object of the Employee class
9  semmy = Employee()
10
11 # call method
12 semmy.showInfo()
```

Python will provide a default constructor if no constructor is defined. Python adds a default constructor when we do not include the constructor in the class or forget to declare it. It does not perform any task but initializes the objects. It is an empty constructor without a body.

Non-parameterized Constructor

```
1  # create class
2  class Employee:
3
4      # no-argument constructor
5      def __init__(self):
6          self.name = "Semmy" # static value
7          self.address = "Bitung City" # static value
8
9      # method print data
10     def showInfo(self):
11         print('Name:', self.name, 'Address:', self.address)
12
13     # create object of the class
14     semmy = Employee()
15
16     # call instance method using the object
17     semmy.showInfo()
```

A constructor without any arguments is called a non-parameterized constructor. This type of constructor is used to initialize each object with default values. This constructor doesn't accept the arguments during object creation. Instead, it initializes every object with the same set of values.

Parameterized Constructor

```
1 # create class
2 class Employee:
3     # parameterized constructor
4     def __init__(self, name, age, salary):
5         self.name = name
6         self.age = age
7         self.salary = salary
8
9     # display data employee
10    def showInfo(self):
11        print(self.name, self.age, self.salary)
12
13    # create object of the Employee class
14    emma = Employee('Semmy Taju', 23, 7000000)
15    emma.showInfo()
16
17    kelly = Employee('Kelly Tanos', 25, 8500000)
18    kelly.showInfo()
```

A parameterized constructor can have any number of arguments.

A constructor with defined parameters or arguments is called a parameterized constructor.

We can pass different values to each object at the time of creation using a parameterized constructor.

The first parameter to constructor is self that is a reference to the being constructed, and the rest of the arguments are provided by the programmer.

Multiple Constructor in Python ?

```
# create class
class Employee:
    # constructor #1
    def __init__(self, name):
        self.name = name

    # constructor #2
    def __init__(self, name, age):
        self.name = name
        self.age = age

    # constructor #3
    def __init__(self, name, age, salary):
        self.name = name
        self.age = age
        self.salary = salary

    # display data employee
    def showInfo(self):
        print(self.name)

# create object of the Employee class
test1 = Employee('Semmy Taju')
test1.showInfo()

test2 = Employee('Juan WaleLang', 25, 8500000)
test2.showInfo()

test3 = Employee('Patricia Mokodompit', 25, 8500000)
test3.showInfo()
```

Is it
possible to
define multiple
constructors in
Python?

Single Constructor *with* Default Parameters

It is not possible to define multiple constructors in Python. However, we can define multiple default values (*parameter values*) if one is not passed.

```
# create class
class Employee:
    # Parameterized constructor with default parameter values
    def __init__(self, name="Bryan Dadidu", age="25", salary=8000000):
        self.name = name
        self.age = age
        self.salary = salary

    # display data employee
    def showInfo(self):
        print("Name: {0}.".format(self.name));
        print("Age: {0} years old.".format(self.age));
        print("Salary: Rp. {0}.".format(self.salary));
        print("-----");

# create multiple objects of the same class
test1 = Employee()
test1.showInfo()

test2 = Employee('Semmy Taju')
test2.showInfo()

test3 = Employee('Juan Walelang', 25)
test3.showInfo()

test4 = Employee('Patricia Mokodompit', 40, 6500000)
test4.showInfo()
```



Exercise *for* Students

Exercise #1 (Constructor & Destructor)

Spyder (Python 3.9)

File Edit Search Source Run Debug Consoles Projects Tools View Help

...KLAB CLASSES\UNKLAB TEACHING\CLASS\Object-Oriented Programming\CODES\Lecture6 [Constructor]\Demo1.py

Demo1.py

```
1 # create class
2 class Student:
3
4     # constructor (initialize instance variable)
5     def __init__(self, name):
6         print('Inside Constructor')
7         self.name = name
8         print('All variables initialized')
9
10    # destructor
11    def __del__(self):
12        print('Inside destructor')
13        print('Object destroyed')
14
15    # instance Method
16    def show(self):
17        print('Hello, my name is', self.name)
18
19
20 # create object using constructor
21 s1 = Student('Semmy Wellem Taju')
22 s2 = Student('Eugenie Patricia Lamansiang')
23 s3 = Student('Putra Buddy Harwaiyan')
24
25 # call method
26 s1.show()
27
28 # delete object
29 del s1
30
31 # call method
32 s1.show()
33
```

Console 4/A X

```
Inside Constructor
All variables initialized
Inside Constructor
All variables initialized
Inside Constructor
All variables initialized
Hello, my name is Semmy Wellem Taju

In [2]: del s1
Inside destructor
Object destroyed

In [3]: s1.show()
Traceback (most recent call last):

  File "C:\Users\USER\AppData\Local\Temp\ipykernel_6176\2710933988.py", line 1, in <module>
    s1.show()

NameError: name 's1' is not defined

In [4]:
```

IPython console History

LSP Python: ready conda: base (Python 3.9.7) Line 1, Col 1 ASCII CRLF RW Mem 78%

Exercise #2 (Instantiation Process)

The screenshot displays the Spyder Python IDE interface. The left pane shows the source code for `Demo2.py`, which defines a `Teacher` class and creates two instances, `t1` and `t2`. The right pane shows the IPython console output, which includes the execution of the code and a `NameError` message.

Source Code (Demo2.py):

```
1 # create class
2 class Teacher:
3     # instance creator (new empty object)
4     def __new__(cls, *args, **kwargs):
5         print("1. Create a new instance of class Teacher.")
6         return super().__new__(cls)
7
8     # constructor (instance initializer)
9     def __init__(self, name, age):
10         self.name = name
11         self.age = age
12         print("2. Initialize the new instance of class Teacher.")
13         print('My name is {0} ({1} years old)'.format(self.name, self.age));
14
15     # string representator
16     def __repr__(self) -> str:
17         return f"type(self).__name__"(name={self.name}, age={self.age})"
18
19     # destructor (object is deleted or destroyed)
20     def __del__(self):
21         print('3. Object destroyed.')
22
23 # create object using constructor
24 t1 = Teacher('Semmy Taju', 30)
25 t2 = Teacher('Eugenie Lamansiang', 17)
26
27 # delete object
28 del t1
29
```

Console Output:

```
...: t1 = Teacher('Semmy Taju', 30)
...: t2 = Teacher('Eugenie Lamansiang', 17)
1. Create a new instance of class Teacher.
2. Initialize the new instance of class Teacher.
My name is Semmy Taju (30 years old).
1. Create a new instance of class Teacher.
2. Initialize the new instance of class Teacher.
My name is Eugenie Lamansiang (17 years old).

In [7]: del t1
3. Object destroyed.

In [8]: t2
Out[8]: Teacher(name=Eugenie Lamansiang, age=17)

In [9]: del t1
Traceback (most recent call last):

  File "C:\Users\USER\AppData\Local\Temp\ipykernel_1684\101134242.py", line 1, in <module>
    del t1

NameError: name 't1' is not defined

In [10]:
```

The status bar at the bottom indicates: LSP Python: ready, conda: base (Python 3.9.7), Line 1, Col 1, UTF-8, CRLF, RW, Mem 87%.

Exercise #3 (Default Parameters)

The image shows the Spyder Python IDE interface. The left pane displays a Python file named `Demo6.py` with the following code:

```
1 # create class
2 class Employee:
3     # Parameterized constructor with default parameter values
4     def __init__(self, name="Bryan Dadidu", age="25", salary=8000000):
5         self.name = name
6         self.age = age
7         self.salary = salary
8
9     # display data employee
10    def showInfo(self):
11        print("Name: {0}.".format(self.name));
12        print("Age: {0} years old.".format(self.age));
13        print("Salary: Rp. {0}.".format(self.salary));
14        print("-----");
15
16    # create multiple objects of the same class
17    test1 = Employee()
18    test1.showInfo()
19
20    test2 = Employee('Semmy Taju')
21    test2.showInfo()
22
23    test3 = Employee('Juan Walelang', 25)
24    test3.showInfo()
25
26    test4 = Employee('Patricia Mokodompit', 40, 6500000)
27    test4.showInfo()
28
```

The right pane shows the console output for the execution of `Demo6.py`:

```
In [4]: runfile('D:/UNKLAB CLASSES/UNKLAB TEACHING/CLASS/Object-Oriented Programming/CODES/Lecture6 [Constructor]/Demo6.py',
wdir='D:/UNKLAB CLASSES/UNKLAB TEACHING/CLASS/Object-Oriented Programming/CODES/Lecture6 [Constructor]')
Name: Bryan Dadidu.
Age: 25 years old.
Salary: Rp. 8000000.
-----
Name: Semmy Taju.
Age: 25 years old.
Salary: Rp. 8000000.
-----
Name: Juan Walelang.
Age: 25 years old.
Salary: Rp. 8000000.
-----
Name: Patricia Mokodompit.
Age: 40 years old.
Salary: Rp. 6500000.
-----

In [5]:
```

The status bar at the bottom indicates: LSP Python: ready, conda: base (Python 3.9.7), Line 1, Col 1, UTF-8, CRLF, RW, Mem 79%.

Modifikasi source code pada file Demo1.py, Demo2.py dan Demo3.py:

1. Dalam masing-masing **class Student, Teacher dan Employee**, tambahkan 1 *class variable* (***status***) dengan type String.
2. Dalam masing-masing **class Student, Teacher dan Employee** buat/tambah 2 method baru (*set_status()* dan *get_status()*). Dimana method *set_status()* memiliki parameter untuk *assign value* ke *variable status*. Dan method *get_status()* tidak memiliki parameter tapi dapat mengembalikan nilai dari *variable status* dengan type String.
3. Method *set_status()* digunakan untuk menginput *value* dari luar class dan harus menggunakan object dari class.
4. Tampilkan output dari method *get_status()* menggunakan object dari masing-masing class.

END PRESENTATION

Thank you for your attention

Instructor: S – W – T

THANK YOU

